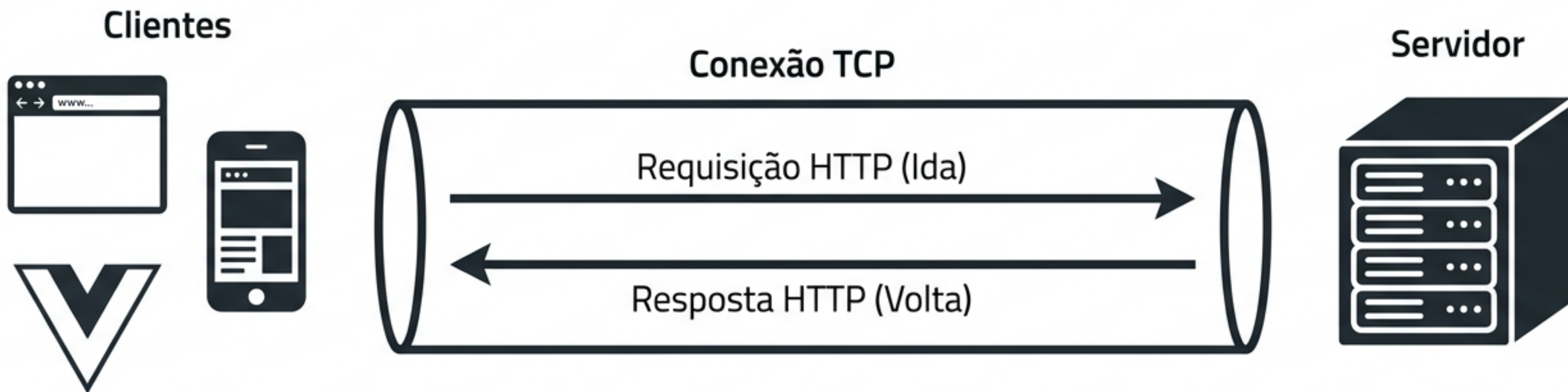




Aula 5.1: Detalhes do HTTP em comunicação com API REST

Prof. Ronaldo Borges

O Ambiente: A Arquitetura Cliente-Servidor



O Motor da Web: O protocolo HTTP opera estritamente sob um modelo de requisição e resposta.

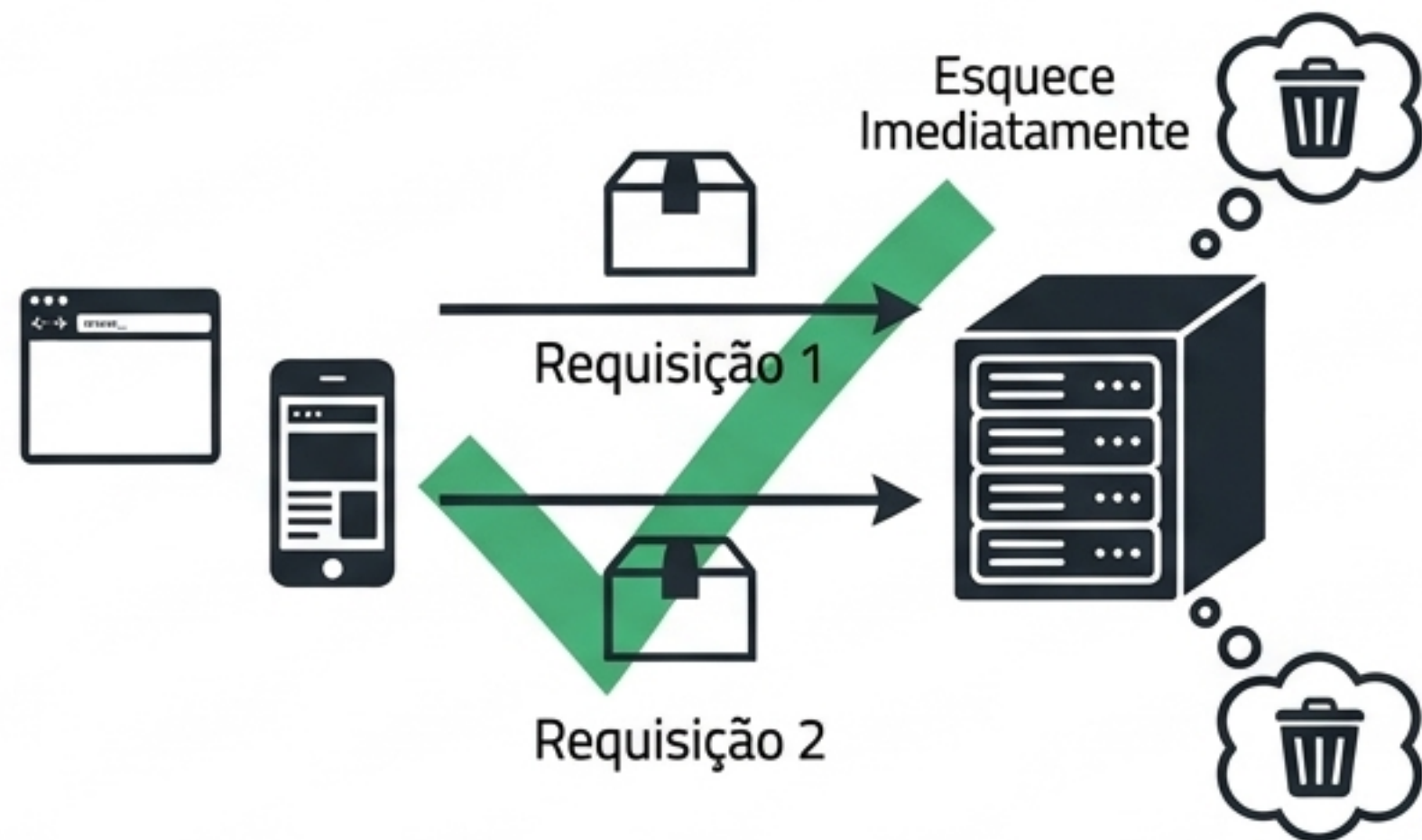
A Regra Prática: A aplicação cliente sempre toma a iniciativa. O servidor processa, executa a regra de negócio e devolve um resultado.

A Regra de Ouro: Comunicação Stateless (Sem Estado)

Incorreto / Stateful



Correto / RESTful



- Cada chamada é absolutamente independente.
- A requisição deve conter todas as informações necessárias (tokens, IDs, dados) para ser compreendida. O servidor não guarda contexto ou estado da sessão entre as chamadas.

A Dinâmica: Semântica das URLs

`https://api.loja.com/pedidos`

Substantivo no plural
representando o Recurso.

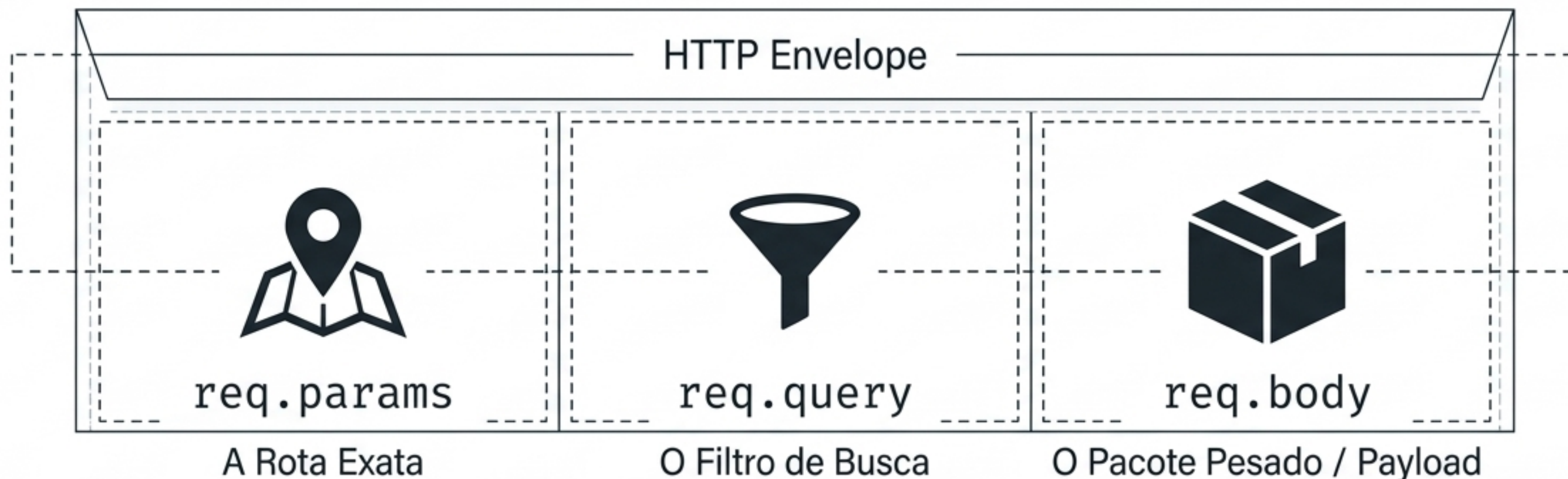
- Rotas e endpoints devem focar em recursos (entidades do sistema) e não em ações.
 - **✗** Evite: `/cadastrarUsuario` ou `/deletarPedido`.
- Quem define a ação que será realizada não é a URL, mas sim o Verbo HTTP.

Mapeamento de Operações:

Matriz CRUD vs. HTTP

Operação (CRUD)	Verbo HTTP	Ação no Servidor	Exemplo de Rota
Create	POST	Cria um novo recurso a partir de um payload	/users
Read	GET	Solicita/recupera informações sem alterar estado	/users ou /users/123
Update	PUT	Atualiza ou substitui integralmente um recurso existente	/users/123
Delete	DELETE	Solicita a remoção/exclusão de um recurso	/users/123

A Anatomia da Requisição: Passagem de Dados



Diferentes necessidades de negócio exigem diferentes métodos de empacotar e enviar dados do cliente para o servidor REST.

1. Parâmetros de Rota (req.params)

api.loja.com/users/**45** →

Identificadores obrigatórios
passados diretamente na
estrutura da URL para localizar
um recurso específico.

```
app.get('/users/:id', (req, res) => {  
  const id = req.params.id; // Retorna 45  
  // Busca usuário específico no banco de dados...  
});
```

2. Parâmetros de Busca (req.query)

api.loja.com/users?cargoc=gerente&ativo=true

Parâmetros opcionais de chave-valor passados após a ?, utilizados para filtragem, busca ou paginação de resultados.

```
app.get('/users', (req, res) => {  
  const cargo = req.query.cargoc; // Retorna gerente  
  const ativo = req.query.ativo; // Retorna true  
  
  // Filtra a lista de usuários baseada nos parâmetros...  
});
```

3. O Payload / Corpo (req.body)



Um pacote oculto que viaja dentro do envelope HTTP, invisível na barra da URL. Usado primariamente para POST e PUT.

O Cliente envia

```
{  
  nome: Ana,  
  idade: 28  
}
```



O Servidor recebe

```
app.post('/users', (req, res) => {  
  const novoUsuario = req.body;  
  console.log(novoUsuario.nome); // Retorna Ana  
  // Salva os dados no banco...  
});
```

Cabeçalhos (Headers) e Negociação de Conteúdo

Cabeçalhos são metadados que ditam as regras da comunicação. A Negociação de Conteúdo permite que cliente e servidor combinem o formato de dados trocados.



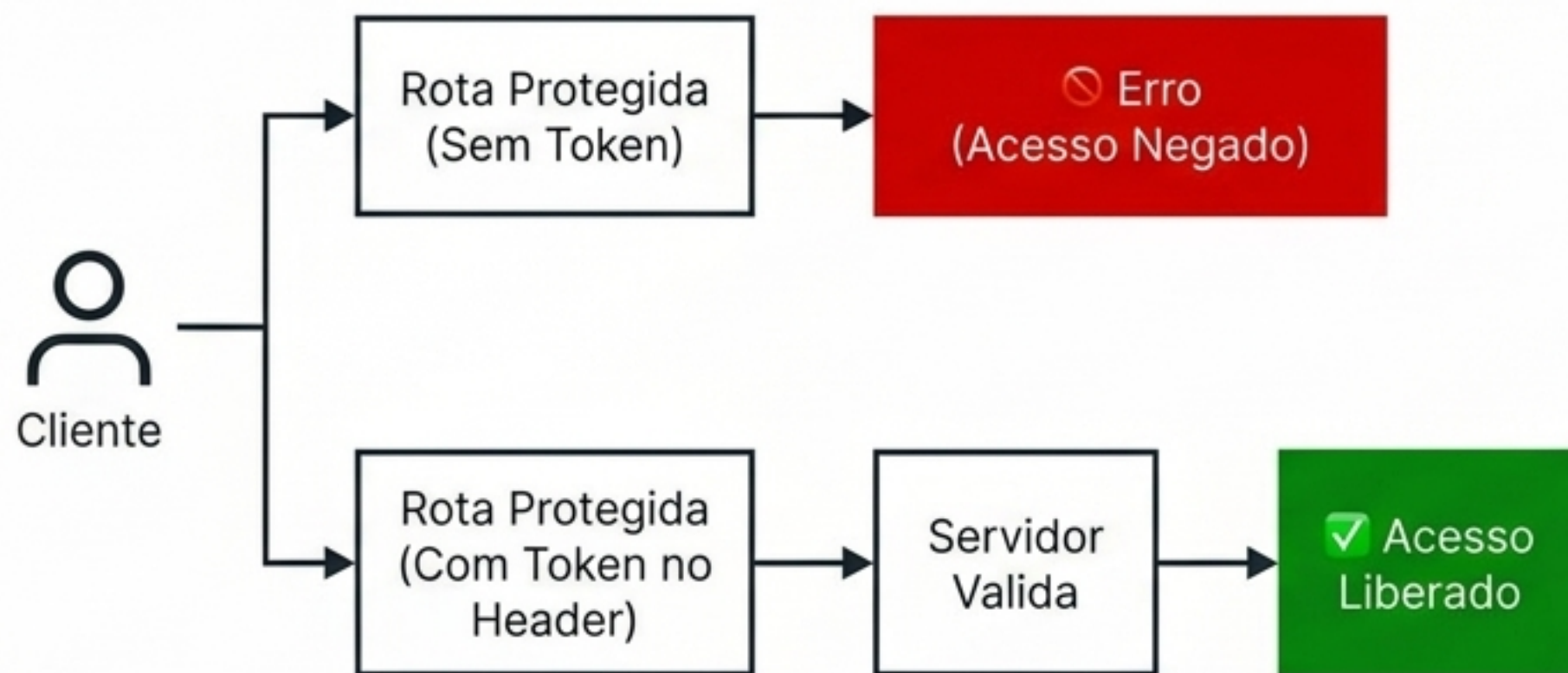
Content-Type:

O tipo de mídia contido no corpo desta requisição (ex.: application/json).

Accept:

O formato de resposta que eu espero e compreendo receber (ex.: application/json).

Cabeçalhos na Prática: Autenticação



Como as APIs REST são estritamente **Stateless**, o cliente deve **provar** sua identidade enviando suas **credenciais** em todas as requisições protegidas.

```
GET /perfil HTTP/1.1
Host: api.loja.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5c...
```

Respostas do Servidor: Códigos de Status (Status Codes)

2xx (Sucesso - A requisição funcionou)

● **200 OK:** A requisição foi processada com sucesso absoluto.

● **201 Created:** Sucesso, e um novo recurso foi efetivamente criado (Ideal para retorno de POST).

4xx (Erros do Cliente - A requisição falhou por culpa do remetente)

● **400 Bad Request:** Requisição inválida, malformada ou falha na validação de dados.

● **401 Unauthorized:** Falta de autenticação. (Quem é você?)

● **403 Forbidden:** Acesso negado. (Eu sei quem é você, mas você não tem permissão).

● **404 Not Found:** O recurso ou rota solicitada não existe no servidor.

A Evolução REST: O Princípio HATEOAS



Sem HATEOAS

O cliente depende de uma lista fixa de URLs chumbadas em seu código.

(Metáfora: Dirigir às cegas com um mapa desatualizado).



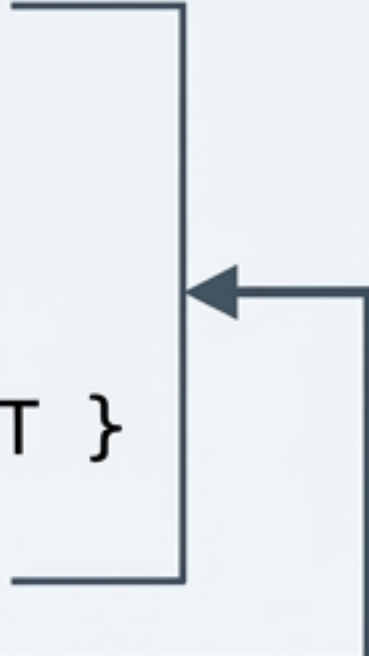
Com HATEOAS

O cliente acessa uma URL raiz e recebe links dinâmicos na resposta, orientando sobre as ações disponíveis a partir daquele estado. O cliente não adivinha rotas.

**HATEOAS: Hypermedia As The Engine
Of Application State.**

Anatomia de uma Resposta HATEOAS

```
{  
  id: 12345,  
  status: Pendente,  
  valor_total: 100.00,  
  links: [  
    { rel: self, href: /pedidos/12345, method: GET },  
    { rel: atualizar, href: /pedidos/12345, method: PUT },  
    { rel: cancelar, href: /pedidos/12345, method: DELETE },  
    { rel: pagar, href: /pedidos/12345/pagamento, method: POST }  
  ]  
}
```



O objeto retorna seus próprios dados junto com o seu manual de instruções de ações possíveis em tempo real. O cliente sabe exatamente como operar sem acoplamento.

Referências Bibliográficas

BROWN, Ethan. Programação web com Node e Express: Beneficiando-se da Stack JavaScript. São Paulo: Novatec Editora, 2020.

DUARTE, Luiz. Programação Web com Node.js: Completo, do Front-end ao Back-end. Edição do Autor (Kindle).

MDN WEB DOCS. JavaScript Docs. Disponível em: <<https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>>. Acesso em: [Data Omitida para o Template].

OPENJS FOUNDATION. Express API Reference. Disponível em: <<http://expressjs.com/pt-br/>>. Acesso em: [Data Omitida para o Template].